

Unit 2

Lesson 11

Overloading

Objective

- Concept of function overloading

“By now you all must have got a fair idea of how a function and its subsequent components viz., arguments/parameters play an important part when used in a C++ program. We also discussed about function returning value by referring ‘directly to the variable’ called as ‘By Val’ and returning value by referring ‘through the address of the variable’ called as ‘By Ref’. The topic, which we are covering, now, is an important and unique feature of object-oriented programming. Lets explore it now....”

FUNCTION OVERLOADING

Overloading refers to the use of the same thing for different purposes. C++ also permits overloading of functions. This means that we can use the same function name to create functions that perform a variety of different tasks. This is known as *function polymorphism* in OOP. The ability to have various functions with the same name in the same program is called **function overloading**. The most important rule about function overloading is to make sure that each one of these functions has a different number or different type(s) of arguments.

Using the concept of function overloading; we can design a family of functions with one function name but with different argument lists. The function would perform different, operations depending on the argument list in the function call. The correct function to invoked is determined by checking the number and type of the arguments but not on the function type. For example, an overloaded add() function handles different types of data shown below:

// Declarations

```
int add(int a, int b);           // prototype 1
int add(int a, intb, int c);     // prototype 2
double add (double x, double y); // prototype 3
double add(int p, double q);     // prototype 4
```

```
double add(double p, int q);           // prototype 5
```

// *Function calls*

```
cout « add(5, 10);                     // uses prototype 1
```

```
cout « add(15, 10.0);                  // uses prototype 2
```

```
cout « add(12.5, 7.5);                 // uses prototype 3
```

```
cout « add(5, 10, 15);                 // uses prototype 4
```

```
cout « add(0.75, 5);                   // uses prototype 5
```

A function call first matches the prototype having the same number and type of argument and then calls the appropriate function for execution. A best match must be unique. The function

Selection involves following steps

1. The compiler first tries to find an exact match in which the types of actual argument are the same, and use that function.
2. If an exact match is not found, the compiler uses the integral promotions to the actual argument, such as,

char to int

Float to double

- d. When either of them fails, the compiler tries to use the built-in conversions(the implicit assignment. conversion) to the actual arguments and then uses function whose match is unique. If the conversion is possible to have multimatches, then the compiler will generate an error message.
- e. Suppose we use following two functions:

long square(long n)

double square (double x)

A function call such as

square(10)

will cause an error because **int** argument can be converted to either **long or double**, thereby creating an ambiguous situation as to which version of **square()** should be used.

4. If all of the steps fail, then the compiler will try the user-defined conversions in combination with integral promotions and built-in conversions to find a unique match. User-defined conversions are often used in handling class objects.

Program illustrates function overloading.

// Function volume() is overloaded three times I

```
#include <iostream>
```

```
using namespace std;
```

```
// Declaration (prototypes)
```

```
int volume(int);
```

```
double volume( double, int) ;
```

```
long volume (long, int , int);
```

```
int main()
```

```
(
```

```
    cout << volume(10) << "\n";
```

```
    cout << volume(2.5,8) << "\n";
```

```
    cout << volume(100L,75,15) << "\n";
```

```
    return 0;
```

```
}
```

```
// Function definitions
```

```
int volume(int s) // cube
```

```

(
    return(s*s*s);
}

double volume(double r, int h)    //cylinder
{
    return (3.14519,*r*r*h) ;
}

long volume (long l, int b, int h) //rectangular box
{
return (l*b*h);
}

```

The output of program:

```

1000
157.26
112500

```

Overloading of function should be done with caution. We should not overload unrelated functions and should reserve function overloading for function that performs closely related tasks. Sometime, the default arguments may be used instead of overloading. This may reduce the number of functions to be defined.

Summary

- The ability to have various functions with the same name in the same program is called function overloading. The most important rule about function overloading is to make sure that each one of these functions has a different number or different type(s) of arguments.
- Overloading of function should be done with caution. We should not overload unrelated functions and should reserve function overloading for function that performs closely related tasks.

Questions

1. Explain the concept of Function Overloading with example
2. Write overloaded function to convert an **int** to an **ascii string** and to convert a **float** to an **ascii string**?
3. Write overloaded function to convert an **ascii string** an **int** and to convert an **ascii string to a float**?